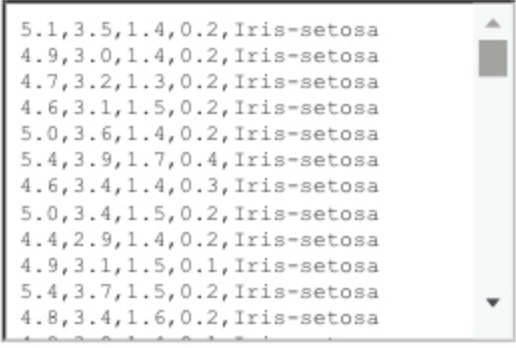


```
In [1]: from IPython.display import HTML
HTML('<iframe src = https://archive.ics.uci.edu/
ml/machine-learning-databases/iris/iris.d
ata width =300, height = 200></iframe>')

Out[1]:
```



```
In [2]: from sklearn.datasets import load_iris

In [3]: iris = load_iris()
type(iris)

Out[3]: sklearn.datasets.base.Bunch

In [4]: print(iris.data)

[[ 5.1  3.5  1.4  0.2]
 [ 4.9  3.  1.4  0.2]
 [ 4.7  3.2  1.3  0.2]
 [ 4.6  3.1  1.5  0.2]
 [ 5.  3.6  1.4  0.2]
```

Figure 1: Code for Jupyter notebook 1

other frameworks of choice. I have used Jupyter for all the code examples.

Building a model

Follow the steps shown below to build a model.

Step 1: For simplicity and ease of demonstration, the iris data set from *kaggle.com* is used here. Kaggle is one of the most popular websites and hosts a lot of data for machine learning model-building (<https://www.kaggle.com/jchen2186/machine-learning-with-iris-dataset>).

This data is a classification of the iris flower along with all its varieties, which are classified based on the length and width of their petals and sepals.

- There are 150 observations with four features each (sepal length, sepal width, petal length and petal width).
 - There are no null values, so we don't have to worry about that.
 - There are 50 observations for each species (setosa, versicolor, virginica, etc).
 - The response variable is the iris species.
 - This is a typical classification problem, as the response is categorical in nature.
 - Import the iris data set from the archive <https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data>
 - Load the 'sklearn' library.
 - Load the data set to 'sklearn' library.
- Please refer to Figure 1.

Step 2: Once the available data is collected, the algorithm to be chosen has to be decided. In classifying

```
In [5]: print (iris.target_names)

['setosa' 'versicolor' 'virginica']

In [6]: print (type(iris.data))

<class 'numpy.ndarray'>

In [8]: print (type(iris.target))

<class 'numpy.ndarray'>

In [9]: print (iris.data.shape)

(150, 4)

In [10]: print (iris.target.shape)

(150,)

In [11]: X = iris.data
y= iris.target
```

Figure 2: Code for Jupyter notebook 2

the iris flower, the KNN K-Nearest Neighbour algorithm will be used.

The following are the requirements to work with the 'scikit-learn' library:

- Feature and response are separate objects
- Feature and response should be numeric
- Feature and response should be NumPy arrays
- Feature and response should have specific shapes

Variable 'x' is a two-dimensional array and variable 'y' is single dimensional. Please refer to Figure 2.

Step 3: As described earlier, the KNN algorithm has been selected and this involves the following steps:

- Pick a value for 'k'.
- Search for the k-observation in the mentioned training data.
- Use the most popular response value from the k-nearest neighbour.

Please refer to Figure 3.

The following are the important steps and measures that are to be taken, while coding this:

- Import the 'estimator'.
- Have the object ready and instantiate it (this is also called 'hyper parameter').
- Fit the model (with feature and response).
- Finally, predict the response from the observation.

Please see Figure 4.

Note: The output of Line-16, which is 'array([2])' is the encoding for the 'virginica' for the unknown Iris. In the Jupyter notebook, the code in Line-30 "DeprecationWarning" can be ignored. In the Jupyter notebook, Output Line-17 'array([2,1])' is the classification for 'Iris_setosa'.